Introduction to Python

Week 6

Program for This Week

1. Comprehensions
2. Generators
3. Handling dates and times with the datetime module
4. On Friday we finish the slides if needed and if we have time left, we can do some questions of the trial exam

Comprehensions

- *Comprehensions* are just a very concise way to write a for loop
- You don't learn new logic here, but you learn a new way to write your loops
- Everything you can do with a comprehension you can do with a loop
- I also will show two examples of a loop and a comprehension doing the same, and use colors, so you can see you are passing Python the same information

List comprehensions

- Example: you have a list called `l1` with integers and you must write code that creates a new list where all integers are ignored when they are divisible by 3, are doubled when they are odd and not divisible by 3, and halved when they are even and not divisible by 3.
  The new list should only contain integers and be called `l2`.
  - For example, `l1 = [1, 3, 2, 4]` should result in `l2 = [2, 1, 2]`
- Based on what you have learned so far, your solution could be:

```
l2 = []
for x in l1:
    if x % 3 == 0:
        continue
    addition = x * 2 if x%2 == 1 else x // 2
    l2.append(addition)
```

- With a comprehension, the solution could be:

```
l2 = [x * 2 if x%2 == 1 else x // 2 for x in l1 if x % 3 != 0]
```

List comprehensions

- These options look very different, but both code fragments contain the same information for Python
- ```
  l2 = []
  for x in l1:
      if x % 3 == 0:
          continue
      addition = x * 2 if x%2==1 else x // 2
      l2.append(addition)
  ```
- ```
  l2 = [x * 2 if x%2==1 else x // 2 for x in l1 if x % 3 != 0]
  ```
- This tells Python, what the name of the new list should be
- This tells Python that we want to create a list, and append the results of the processed elements to that list
- This tells Python what the source is of which we want to process the elements
- This tells Python, which elements to include for processing (be careful in this set-up of the for loop you tell what to exclude but in a comprehension what to include)
- This tells Python, what to do with the elements that should be processed

Dictionary comprehensions

- Example: you have two lists called `l1, l2` with integers and you must write code that creates a dictionary where the keys come from `l1` and the values are triple those from `l2`, except when the key and/or the value can be divided by 2, then these values should be ignored.
  The new dictionary should be called `d1`.
  - For example, `l1 = [1, 3, 2, 5], l2 = [1, 2, 6, 3]` should result in `d1 = {1:3, 5:9}`
- Based on what you learned so far, the solution could be:

```
d1 = {}
for k,v in zip(l1,l2):
    if k%2 == 0 or v%2 == 0:
        continue
    d1[k] = v * 3
```

- With a comprehension, the solution could be:

```
d1 = {k: v*3 for k,v in zip(l1,l2) if not (k%2 == 0 or v%2 == 0)}
```

Dictionary comprehensions

- ```
  d1 = {}
  for k,v in zip(l1,l2):
      if k%2 == 0 or v%2 == 0:
          continue
      d1[k] = v * 3
  ```
- ```
  d1 = {k: v*3 for k,v in zip(l1,l2) if not (k%2 == 0 or v%2 == 0)}
  ```
- This tells Python, what the name of the new list should be
- This tells Python that we want to create a dictionary, and append the results of the processed elements to that dictionary
- This tells Python what the source is of which we want to process the elements
- This tells Python, which elements to include for processing (be careful in this set-up of the for loop you tell what to exclude but in a comprehension what to include)
- This tells Python, what to do with the elements that should be processed

Set comprehensions

- Example: you have a list called `l1` with integers and you have to write code that creates a set, with all the values from the list doubled, while ignoring values from the list that are even.
  The new set should be called `s1`.
- For example, `l1 = [1, 3, 2, 5]` should result in `s1 = {2, 6, 10}`
- Based on what you learned so far, the solution could be:

```
s1 = set()
for x in l1:
    if x%2 == 0:
        continue
    s1.add(x * 2)
```

- With a comprehension, the solution could be:

```
s1 = {x * 2 for x in l1 if not (x%2 == 0)}
```

- Question: Why does Python understand you are building a set and not a dictionary?

Set comprehensions

- Example: you have a list called `l1` with integers and you have to write code that creates a set, with all the values from the list doubled, while ignoring values from the list that are even. The new set should be called `s1`.
- For example, `l1 = [1, 3, 2, 5]` should result in `s1 = {2, 6, 10}`
- Based on what you learned so far, the solution could be:

```
s1 = set()
for x in l1:
    if x%2 == 0:
        continue
    s1.add(x * 2)
```

- With a comprehension, the solution could be:

```
s1 = {x * 2 for x in l1 if not (x%2 == 0)}
```

- Question: why does Python understand you are building a set and not a dictionary?
- Answer: because you don't create key: value pairs, but single values

'string comprehensions'

- Example: you have a string called `s1` with only letters and digits and you have to write code that creates a string where all vowels doubled, all consonants tripled, and all digits ignored. The new string should be called `s2`
- For example, `s1 = 'a1BcDE2FgH3i'` should result in
  `s2 = 'aaBBBcccDDDEEFFFgggHHHii'`
- Based on what you learned so far, the solution could be:

```
import string
s2 = ''
for x in s1:
    if x in string.digits:
        continue
    s2 += x*2 if x.lower() in 'aeiou' else x*3
```

- Question: string comprehensions don't exist; so how would you solve this problem with a comprehension?
  Hint: Remember how we sorted a string.

'string comprehensions'

- With a comprehension, the solution could be:

```
import string
l1 = [x * 2 if x.lower() in 'aeiou' else x * 3 for x in s1 if x in
string.ascii_letters]
```

- `s2 = ''.join(l1)`
- So, you use a list comprehension to solve the question
- There is actually no such thing as a string comprehension, and therefore I put quotes around it

Nested comprehensions

- Example: you want to create a dictionary `result`, with keys being the integers between 1 and 4, and values are sub-dictionaries. The sub-dictionaries have keys that start with the same integer as the key in result till 4 and the values are sum of these keys.
- For example:
  `print(result)` should give : `{1: {1: 2, 2: 3, 3: 4}, 2: {2: 4, 3: 5}, 3: {3: 6}}`
  `print(result[2][3])` should give `5`
- `func = lambda x,y: x+y`
- ```
  result = {}
  for x in range(1, 4):
      result1 = {}
      for y in range(x, 4):
          result1[y] = func(x,y)
      result[x] = result1
  ```
- With a comprehension, the solution could be:
- `result = {x:{y: func(x,y) for y in range(x, 4)} for x in range(1, 4)}`
- Read the comprehension right to left
  ```

Nested comprehensions

- Example: you want to create a dictionary `result`, with keys being tuples of two integers (second one greater equal) between 1 and 4, and values that are the sum of these numbers.
- For example:
  `print(result)` should give :
  `{(1, 1): 2, (1, 2): 3, (1, 3): 4, (2, 2): 4, (2, 3): 5, (3, 3): 6}`
  `print(result[(2, 3)]` should give `5`
- `func = lambda x,y: x+y`
- ```
  result = {}
  for x in range(1, 4):
      for y in range(x, 4):
          result[x,y] = func(x,y)
  ```
- With a comprehension, the solution could be:
- `result = {(x, y): func(x,y) for x in range(1, 4) for y in range(x, 4)}`
- Read the comprehension from right to left
  ```

List comprehensions (old school)

- In week 3 Hugo explained, in one of the Datacamp video's, lambda functions in combinations with map and filter. This combination also can make your code very concise
- I will show you how that works, and I hope you will agree, that comprehensions are the better option
- Say you have a list called `l1` with integers and you have to write code that creates a new list where all integers are ignored when they are divisible by 3, are doubled when they are odd and not divisible by 3, and halved when they are even and not divisible by 3.
  The new list should only contain integers and be called `l2`
- For example, `l1 = [1, 3, 2, 4]` should result in `l2 = [2, 1, 2]`
- With a comprehension, the solution could be:
  ```
  l2 = [x * 2 if x%2 == 1 else x // 2 for x in l1 if x % 3 != 0]
  ```
- With map, filter and lamba, the solution could be:
  ```
  l2 = list(map(lambda x: x * 2 if x%2 == 1 else x//2, filter(lambda x: x % 3 != 0, l1)))
  ```
- There is a reason map and filter are used less since the introduction of comprehensions in Python

How to read this

- `l2 = list(map(lambda x: x * 2 if x%2 == 1 else x//2, filter(lambda x: x % 3 != 0, l1)))`
- Start at the deepest level wrt the brackets
- `filtered_values = filter(lambda x: x % 3 != 0, l1)`
- `processed_values = map(lambda x: x * 2 if x%2 == 1 else x//2, filtered_values)`
- `list_of_processed_values = list(processed_values)`
- `print(list_of_processed_values)`

Question

We already have the following code:

```
s1 = '01212321'
set1 = {int(x) * 2 for x in s1 if x in '02468'}
```

Which of the following lines of code will print True?

a. `print(set1 == {'0', '4', '4', '4'})`

b. `print(set1 == {4, 0})`

c. Both will print `True`

d. Neither one will print `True`

Question

We already have the following code:

```
s1 = '01212321'
set1 = {int(x) * 2 for x in s1 if x in '02468'}
```

Which of the following lines of code will print True?

a. `print(set1 == {'0', '4', '4', '4'})`

b. `print(set1 == {4, 0})`

c. Both will print `True`

d. Neither one will print `True`

e. Cannot be correct as the created elements are clearly integers

f. Is correct. Sets only keep unique values, and there is no order
Actually print(set1 == {4, 4, 4, 4, 0}) would also lead to True, as the created set object will only keep two unique values: 0 and 4

Keeping state between function calls

- Assume, you must write a function that returns a certain start value, and next times you call the function returns the previous value plus a certain step value, until the value would be greater than a certain stop value, in which case the function throws an error
- For example, if you have a start value = 1, a stop value of 5, and a step value of 2,
- The first time that function is called, it should return 1, the second time 3, the third time 5, and the fourth time it should throw an error
- This looks like a very simple problem. Adding 2 to a number is not that complicated, checking whether a number is greater than the stop value also looks very doable
- The problem is that in between function calls you need to keep track of the value of the last number. (This is called keeping track of the state.)
- After execution, a function and its local names will all have disappeared
- How to solve this?

1. Using a Standard function
2. Using a Standard function and the global keyword
3. Using Object oriented programming
4. Using an iterator
5. Using a generator function

Keeping state with a standard function

- ```
  def myfunc(start, stop, step):
      num = start
      start += step
      if num > stop:
          raise StopIteration('End of numbers')
      else:
          return num, start, stop, step
  ```
- ```
  sta = 1
  sto  = 5
  ste  = 2
  ```
- ```
  num, sta, sto, ste = myfunc(sta, sto, ste)
  print(num)        # Onscreen: 1
  ```
- ```
  num, sta, sto, ste = myfunc(sta, sto, ste)
  print(num)        # Onscreen: 3
  num, sta, sto, ste = myfunc(sta, sto, ste)
  print(num)        # Onscreen: 5
  ```
- ```
  num, sta, sto, ste = myfunc(sta, sto, ste)
                    # Onscreen: StopIteration: End of numbers
  ```
- This solution is a bit inconvenient as you must to store all values between calls yourself
- If we want to store behavior and data together, using a class looks like a good solution
  ```

Keeping state with a standard function and the global keyword

- ```
  def myfunc():
      global start, stop, step
      num = start
      start += step
      if num > stop:
          raise Exception('End of numbers')
      else:
          return num
  ```
- ```
  start = 1
  stop  = 5
  step  = 2
  ```
- ```
  print(myfunc())  # Onscreen: 1
  print(myfunc())  # Onscreen: 3
  print(myfunc())  # Onscreen: 5
  print(myfunc())  # Onscreen: Exception: End of numbers
  ```
- This solution is a bit inconvenient as you must store the data outside the function, and use the global keyword
- If we want to store behavior and data together, using a class looks like a good solution
  ```

Keeping state with a standard class

- ```
  class Myfunc:
      def __init__(self, start, stop, step):
          self.start = start
          self.stop = stop
          self.step = step
  ```
- ```
      def following(self):
          self.num = self.start
          self.start += self.step
          if self.num > self.stop:
              raise Exception('End of numbers')
          else:
              return self.num
  ```
- ```
  myfunc = Myfunc(1, 5, 2)
  ```
- ```
  print(myfunc.following())    # Onscreen: 1
  print(myfunc.following())    # Onscreen: 3
  print(myfunc.following())    # Onscreen: 5
  print(myfunc.following())    # Onscreen: Exception: End of numbers
  ```
- This works, but Python offers a more dedicated solution: iterators

Keeping state with a dedicated class: iterators

- ```
  class Myiterator:
      def __init__(self, start, stop, step):
          self.start = start
          self.stop = stop
          self.step = step
  ```
- ```
      def __next__(self):
          self.num = self.start
          self.start += self.step
          if self.num > self.stop:
              raise StopIteration('End of numbers')
          else:
              return self.num
  ```
- ```
  myiterator = Myiterator(1, 5, 2)
  ```
- ```
  print(next(myiterator))      # Onscreen: 1
  print(next(myiterator))      # Onscreen: 3
  print(next(myiterator))      # Onscreen: 5
  print(next(myiterator))      # Onscreen: Exception: End of numbers
  ```
- This works well, but sometimes Python offers another nice solution to create iterators: generator functions

Keeping state with a generator function

- `def generatorfunction(start, stop, step):`
-
```
    while (num:=start) <= stop:
        yield num
        start += step
    return 'End of numbers'
```
- `mygenerator = generatorfunction(1, 5, 2) # Creating the generator`
-
```
print(next(mygenerator))     # Onscreen: 1
print(next(mygenerator))     # Onscreen: 3
print(next(mygenerator))     # Onscreen: 5
print(next(mygenerator))     # Onscreen: StopIteration: End of numbers
```

Generator functions

- A generatorfunction is a function that contains a yield statement and that you use to construct a generator
- A yield statement returns a value, stops the function, but Python keeps the function and all its objects in memory (the data of standard functions are all lost after the function execution
- You can compare it with a computer that goes in sleep (or hibernate) mode, if you restart the computer, you will resume where you were, on the moment, when the computer went in sleep mode
- If the generator is called again your program runs the statement that should be executed after the yield statement
- `def generatorfunction():`
-
```
    num = 1
    yield num
    num += 2
    yield num
    num += 2
    yield num
```
- `mygenerator = generatorfunction()          # Creating the generator`
-
```
print(next(mygenerator))      # Onscreen: 1
print(next(mygenerator))      # Onscreen: 3
print(next(mygenerator))      # Onscreen: 5
print(next(mygenerator))      # Onscreen: StopIteration: End of numbers
```

Using a Generator function inside a loop

- `def generatorfunction(start, stop, step):`
-
```
    while (num:=start) <= stop:
        yield num
        start += step
```
- `    return 'End of numbers'`
-
```
for x in generatorfunction(1, 5, 2):
    print(x)     # Onscreen: 1
                            3
                            5
```
- No error here, as the for loop catches that error for you

Turning a generatorfunction into a list

- `def generatorfunction(start, stop, step):`
-
```
    while (num:=start) <= stop:
        yield num
        start += step
```
- `    return 'End of numbers'`
- `print(list(generatorfunction(1, 5, 2))) # Onscreen: [1, 3, 5]`
- So, you can do this, but in a way, defies the idea of a generator
- A generator is very efficient on memory usage, only delivering a next value if asked for it
- Question: As always there is a trade-off, could you guess what?
- Answer: for small lists, the overhead of the generator starts to count

Turning a list into an iterator

- `myiterator = iter([1,3,5])`
- ```
  print(next(myiterator))      # Onscreen: 1
  print(next(myiterator))      # Onscreen: 3
  print(next(myiterator))      # Onscreen: 5
  print(next(myiterator))      # Onscreen: StopIteration
  ```
- Works because the list class has an __iter__ and a __next__ method
  ```

Generator comprehension

- `mygenerator = (x for x in range(100000000) if x%3 == 0 or x%4 == 0)`
- ```
  print(next(mygenerator))     # Onscreen: 0
  print(next(mygenerator))     # Onscreen: 3
  print(next(mygenerator))     # Onscreen: 4
  print(next(mygenerator))     # Onscreen: 6
  ```
- vs
- `l1 = [x for x in range(100000000) if x%3 == 0 or x%4 == 0]`
- ```
  print(l1[0])
  print(l1[1])
  print(l1[2])
  print(l1[3])
  ```

Handling date and times

- Dates and times in Python are measured as seconds since 1970-01-01 00:00:00 UTC Coordinated Universal Time, also known as Greenwich Mean Time (GMT)
- UTC doesn't have daylight saving time, so every day has 24 hours
- To get the time in seconds since that moment:
- ```
  from datetime import datetime
  print(datetime.now().timestamp())
    # On screen: 1716179932.022811
  ```
- This is great for computers, but not so much for humans
  ```

Classes in the datetime module

There are several classes defined in the datetime module, we start with the following three:

1. datetime.date
   This object stores a date and has the year, month, and day as attributes. It is based on the Gregorian calendar, and assumes the world always used the Gregorian calender
2. datetime.time
   This object stores a time and has the hour, minute, second, microsecond, and tzinfo (time zone information) as attributes. It assumes there are 86,400 seconds in every day, so no leap seconds
3. datetime.datetime
   This object is a combination of a datetime.date and a datetime.time. It has all the attributes of both classes

(Here you see, that while the name of user-built classes starts with a capital, the names of built-in classes do not (always))

We will only use the datetime class in these slides. But the idea for datetime.date and datetime.time is the same

Creating a datetime object

- Creating a datetime object by passing integers
  `from datetime import datetime`
- `d = datetime (year=2023, month=5, day=22, hour=17, minute = 10, second=30)`
- `d = datetime (hour=17, month=5, day=22, second=30, minute = 10, year=2023)`
- `d = datetime (2023, 5, 22, 17, 10, 30 )`
- `d = datetime (2023, 5, 22, minute = 10, day = 22, hour = 17, second=30 )`
- `print(d)`
- The order of the positional parameters: `year>month>day>hour>minute>second`

attributes of a datatime object

- `from datetime import datetime`
- `d = datetime.now()`
- `print(f'{d.year = }, {d.month = }, {d.day = }, {d.hour = }, {d.minute = }, {d.second = }, {d.microsecond = }')`
- ```
  print(f'''
  Datum en tijd in ISO format: {d.isoformat()}
  Day of the week (1=Monday): {d.isoweekday()}
  Week of the year: {d.isocalendar()[1]}
  Year: {d.isocalendar()[0]}
  ''')
  ```
- In the next slide you will see that you can get all this info with the help of the strftime method
- The difference is that these attributes give the information as integers, strftime gives it in string format. We can easily use `str` and `int`
  ```

Creating a string representation a datetime object

- `d = datetime.now()`
- `print(d)`
- ```
  print(f'''
  {d.strftime('Year with century: %Y') = }
  {d.strftime('Year without century: %y') = }
  {d.strftime('Full month name: %B') = }
  {d.strftime('Month number: %m') = }
  {d.strftime('Day of the month: %d') = }
  ```
- ```
  {d.strftime('Full weekday name: %A') = }
  {d.strftime('Weekday number: %w') = }
  ```
- ```
  {d.strftime('Week number of the year: %U') = }
  {d.strftime('Day number of the year: %j') = }
  ```
- ```
  {d.strftime('Hour number: %H') = }
  {d.strftime('Minute number: %M') = }
  {d.strftime('Second number: %S') = }
  ```
- `''')`

Creating a datetime object by passing a string

- Say you want to create the following datetime object with date and time: 2024-05-20 00:00:00, the following codes all manage to do this:
- `from datetime import datetime`
- `d = datetime.strptime('2024$05%20','%Y$%m%%%d')`
- `d = datetime.strptime('24$05%20','%y$%m%%%d')`
- `d = datetime.strptime('24$05%20','%y$%m%%%d')`
- `d = datetime.strptime('2024*141','%Y*%j')`
- `d = datetime.strptime('2024*20xyz1','%Y*%Uxyz%w')`
- `d = datetime.strptime('202420Monday','%Y%U%A')`
- `d = datetime.strptime('00:00:00 20-05-2024', '%H:%M:%S %d-%m-%Y')`

Changing a datetime object with the replace method

- You can change all (or some) attributes of a datetime object with the replace method:
- ```
  from datetime import datetime
  d = datetime.now()
  ```
- ```
  d = d.replace(year=2026, month=10, day=13, hour=13, minute=10,second=13, microsecond=123456)
  ```
- You can do more advanced replacements:
- ```
  from datetime import datetime
  ```
- ```
  d = d.replace(year=d.year - 2)
  ```

Arithmetic with datetime and timedelta objects

- If you subtract datetimes a timedelta object is automatically created:
- ```
  from datetime import datetime
  birthday = datetime(1923, 10, 26)
  today = datetime.now()
  ```
- ```
  age = today - birthday
  ```
- ```
  print(f'{age.days}{age.seconds}')  # On screen: 36732 14249
  ```
- ```
  print(type(age))                   # On screen: Timedelta
  ```

Arithmetic with datetime and timedelta objects

- Timedelta has only 3 attributes that you can access: days, seconds and microseconds. So, if you want to calculate the age in years, that is not that simple.
- `age = today.year - birthday.year - 1`
- `age += today.month > birthday.month or today.month == birthday.month and today.day >= birthday.day`

- But if you don't subtract two datetimes, but want to add or subtract for example some days to or from a datetimet, you need to import the timedelta class explicitly, and create a timedelta object:
- `from datetime import timedelta, datetime`
- `d = datetime(2022, 5, 24)`
- `diff = timedelta(days=10)`
- `print(d + diff) # Onscreen: 2022-06-03 00:00:00`
- `print(type(d + diff)) # Onscreen: <class 'datetime.datetime'>`

Arithmetic with datetime and timedelta objects

- If you want to add or subtract days or seconds to a datetime object, the timedelta object can be very convenient.
- While the timedelta object has three attributes accessible, there are more parameters one can use, to define a timedelta object
- The following codes will all give the same result:
- `from datetime import datetime, timedelta`
- `today = datetime.now()`
- `future_day = datetime.now() + timedelta(days=100)`
- `future_day = datetime.now() + timedelta(weeks=14, days=2)`
- `future_day = datetime.now() + timedelta(hours=100*24)`
- `future_day = datetime.now() + timedelta(minutes=100*24*60)`
- `future_day = datetime.now() + timedelta(seconds=100*24*60*60)`